

SPL Playground

How a browser-based Splunk search engine was built

Engineering build report | 11 August 2026

This report describes how the SPL Playground was designed, implemented and verified. It is a single self-contained HTML file that interprets Splunk Search Processing Language entirely in the browser, against generated sample data, with no Splunk instance, no server and no install.

Contents

1. The problem and the constraint	2
2. Architecture	2
3. The SPL engine.....	3
3.1 Two grammars, two parsers	3
3.2 Details that decide whether it feels real.....	3
3.3 The command layer	4
4. Generating data worth searching	4
4.1 Determinism is a requirement, not a nicety.....	4
4.2 Random events teach the wrong lesson	4
5. Grading exercises without demanding one exact answer	5
6. Verification.....	6
6.1 Invariants over golden values	6
6.2 What it found.....	6
7. Interface and packaging.....	7
8. Deliberate limits	7
9. Summary.....	8

1. The problem and the constraint

The requirement was a place to run SPL queries and check understanding of the commands, without standing up Splunk. A trial licence expires, and a local install is a heavy prerequisite for what is fundamentally revision practice.

The build machine settled the architecture. A survey of the available toolchain returned almost nothing:

```
$ node --version    -> command not found
$ npm  --version    -> command not found
$ python --version  -> Microsoft Store alias stub, not a real interpreter
```

With no runtime to build against and no server to deploy to, anything requiring a package manager, a bundler or a backend was ruled out. That pointed at one target: a zero-dependency application that runs entirely client side, ships as a single HTML file, and therefore works offline, works from a file:// URL, and can also be hosted and shared as a link without any infrastructure.

The consequence is that the SPL engine itself had to be written from scratch in JavaScript. That turned out to be the bulk of the work and the most interesting part of it.

2. Architecture

The application is five source files plus a build script. The separation is deliberate: the language core knows nothing about commands, the command layer knows nothing about the UI, and the UI knows nothing about how searches are evaluated.

File	Responsibility	Approx. size
src/data.js	Seeded mock data generator and lookup tables	400 lines
src/spl-lang.js	Tokenizer, expression parser, evaluator, eval function library, time and regex handling	700 lines
src/spl-commands.js	Search parser and matcher, aggregation functions, ~60 commands, pipeline runner	1,500 lines
src/content.js	Command reference, samples, exam notes, graded exercises	450 lines
src/app.js	User interface, charts, exercise grader	900 lines
src/styles.css	Theming for light and dark	400 lines
build.ps1	Concatenates everything into one HTML file	40 lines

Data flows in one direction. The generator produces events once at page load. A search is parsed, the base filter selects events, and the surviving rows pass through each command in turn. Each command receives a result set and returns a new one.

```
runSearch(spl, opts)
|
+-- expandMacros()          resolve `backtick` macros
+-- splitPipeline()         split on | outside quotes and brackets
+-- parseSearch()           base filter -> boolean AST
|   +-- time bounds applied, events matched
+-- applySegment() x N      one command at a time
```

```
State { rows, fields, isEvents, chart } -> State
```

3. The SPL engine

3.1 Two grammars, two parsers

SPL is not one language. The base search is a boolean retrieval grammar where bare words are full-text terms and adjacency means AND. An eval expression is an ordinary infix expression language. Trying to serve both with one parser produces something that handles neither well, so the engine has two.

The search parser tokenizes into atoms, parentheses and bracketed subsearches, then parses with this precedence:

OR	lowest
AND	implicit between adjacent terms
NOT	unary
term	field=value field IN (..) bare full-text term

The expression parser used by eval and where is a recursive-descent parser over a conventional precedence ladder:

OR	
AND / XOR	
NOT	
comparison	= == != < > <= >= LIKE IN
additive	+ - . (. is string concatenation)
multiplic.	* / %
unary	-
primary	literal field function(..) (expr)

3.2 Details that decide whether it feels real

A handful of small decisions do most of the work in making the engine behave the way Splunk does.

The dot is ambiguous. In SPL the dot is the string concatenation operator, but field names such as `http.status` also contain dots. The tokenizer resolves this positionally: a dot glued to identifier characters on both sides is part of the name, otherwise it is the operator. So `http.status` is one field and `host . ":" . status` is a concatenation.

Negation cannot be evaluated per value. A multivalue field matches a positive comparison if any one of its values matches. Applying the same rule to an inequality gives the wrong answer, because "any value is not X" is true whenever the field has two different values. Negation is therefore evaluated against the whole field: no value may match. This is also what separates `NOT status=200`, which keeps events with no status field at all, from `status!=200`, which requires the field to exist.

Some functions must not evaluate their arguments. Evaluating every argument before the call breaks `if()`, `case()`, `validate()` and `coalesce()`, which exist precisely to avoid evaluating branches that should not run. Those are handled as lazy forms before argument evaluation. The same

mechanism supports `mvfilter()`, which has to bind each value of a multivalue field in turn and evaluate the expression once per value.

Null propagates, comparisons do not. Arithmetic involving null yields null, matching Splunk. Comparison against null yields false rather than null, so a where clause cannot accidentally keep rows on a missing field.

3.3 The command layer

Every command has the same shape, which is what keeps a sixty-command library manageable:

```
CMD.<name> = function (State, argString, ctx) -> State

State = {
  rows:    array of row objects
  fields:  explicit column order, or null to derive from the data
  isEvents: are these still raw events, or a statistics table?
  chart:   optional { x, series } hint for the visualiser
}
```

Argument parsing is shared rather than reimplemented per command. Four helpers cover almost every case: a quote-aware and bracket-aware splitter, an option extractor that pulls leading key=value pairs, a clause splitter that separates aggregations from BY and OVER, and an aggregation parser that understands `sum(bytes) AS total` and `count(eval(status=404))`.

`chart` and `timechart` share one implementation. `timechart` is `chart` with `_time` on the x-axis, so it buckets time, delegates, and then fills empty buckets so the series stays continuous. When no span is given the engine picks one from a list of round intervals, choosing the smallest that keeps the chart under about sixty points.

4. Generating data worth searching

The dataset is about 5,970 events across six indexes and six sourcetypes, covering the last seven days: web access logs, Linux SSH authentication, multi-line Windows security events, key=value firewall traffic, vendor sales, and JSON application logs for `spath` practice. Five lookup tables sit alongside them.

4.1 Determinism is a requirement, not a nicety

Generation runs from a seeded pseudo-random generator, a small `mulberry32` implementation, rather than `Math.random`. Every user, and every reload, produces byte-identical data. This is not tidiness: the exercise grader compares a learner query against a reference query, and that comparison is only meaningful if the underlying data cannot shift between the two runs.

4.2 Random events teach the wrong lesson

The first generator emitted independent events, each with a freshly randomised client IP and session id. It looked plausible and was useless. With roughly 7,000 possible session ids spread across 2,600 events, almost nothing repeated, so the queries that matter most returned nothing:

```
index=web | stats count by clientip | where count > 25
-> 0 results
```

```
index=web | transaction JSESSIONID maxspan=30m | where eventcount > 5  
-> 0 results
```

A learner running those and seeing an empty table would reasonably conclude they had written them wrong. The generator was restructured so web traffic is produced as browsing sessions: a fixed pool of ninety client IPs, each session holding one client, one session id and a run of page views a few minutes apart, with realistic page sequencing. SSH sources and firewall endpoints got the same treatment, including four brute-force source addresses that generate a heavy tail of failures.

The same queries now behave as they would against a real access log:

```
index=web | stats count by clientip | where count > 25  
-> 50 rows, top talker 66 events  
  
index=web | transaction JSESSIONID maxspan=30m | where eventcount > 5  
-> 145 sessions
```

This is the single most important decision in the data layer. The realism is not decorative; it is what makes transaction, dedup and per-entity aggregation demonstrate anything.

5. Grading exercises without demanding one exact answer

There are 25 graded exercises. Marking them by comparing query text would be worse than useless, because SPL offers many equivalent phrasings and a learner who finds a different correct route should not be told they are wrong.

Instead the grader executes both queries and compares the results:

1. run the reference solution over the full dataset
2. run the learner query over the same data
3. compare the two result tables

The comparison was tuned against real disagreements found during testing:

Column names are compared as a set, not a sequence, so declaring aggregations in a different order still passes.

Column order is enforced only where an exercise is explicitly about it, via a strictCols flag set on the table-ordering question.

Row order is always compared, because the sorting exercises depend on it.

Values are normalised before comparison: numbers are rounded to six decimal places and multivalue fields are flattened.

Event-level exercises with no meaningful table use a count-only comparison mode.

Failures report the specific difference, naming the row and column that disagree, rather than saying "incorrect".

Grading always runs over all seven days regardless of the time picker, so a correct answer cannot be marked wrong because of an unrelated UI setting. The panel says so explicitly.

6. Verification

A search engine that silently returns the wrong number is worse than one that fails loudly, because a study tool that quietly teaches wrong behaviour is actively harmful. Verification was therefore run against the built bundle in a real browser, not against the source in principle.

6.1 Invariants over golden values

Around eighty assertions were executed inside the running page. Most check properties that must hold rather than values hard-coded from a previous run, which is what lets them detect a regression rather than merely restate one:

Invariant	Why it catches real bugs
<code>count(NOT status=200) + count(status=200) = total</code>	Negation and multivalue handling
Sum of all chart cells = total event count	Split-by bucketing, OTHER handling, limits
Sum of transaction eventcount = total events	No event dropped or double-counted when grouping
timechart bucket boundaries are exactly one span apart	Gap filling and span alignment
<code>avg(x) = sum(x) / count(x)</code>	Aggregation arithmetic
Sum of per-group counts = ungrouped count	Grouping key construction

Alongside those, every example in the command reference, every sample search and every exercise reference answer is executed, and any that errors or returns an empty table is reported. That keeps the documentation honest: a documented example that no longer works fails the run.

6.2 What it found

The suite found five genuine defects, all fixed:

Defect	Effect	Cause
cidrmatch always returned true	Every subnet matched every address, including 192.168.0.0/16 matching 10.1.1.1	The netmask was used as a modulus instead of dividing away the host bits
Quoting destroyed regular expressions	rex "port (?<src_port>\d+)" extracted nothing at all	The unquote routine consumed every backslash escape, turning \d into a literal d
top 5 field misread as a field name	The bare-integer limit form returned no rows	Only the limit= option form was handled
Five NUL bytes in the source	The data reference panel rendered its table incorrectly	Single-space string literals were written as NUL instead of space
Malformed IP addresses	Addresses such as 203.0.113.241.213 were generated	A three-octet prefix had two more octets appended

Four further issues were found by testing alternative correct answers against the grader and by checking the layout at narrow widths: sort with a space between the minus sign and the field name, non-deterministic ordering of tied values in top, over-strict column-order comparison,

and a flexbox rule that let the results pane grow past the viewport on small screens instead of scrolling within it.

The cidrmatch bug is the one worth dwelling on. It never threw an error and never returned an empty table. It simply returned true, so every search using it produced a full, plausible, entirely wrong result. Only an assertion comparing two different subnets exposed it.

7. Interface and packaging

The interface follows the shape of the Splunk search view, because familiarity is part of the point: search bar and time picker, a timeline of matching events, a field sidebar with value breakdowns, and Events, Statistics and Visualization tabs. A study panel carries the command reference, function reference, sample searches, graded exercises, exam notes and a description of the datasets.

Two decisions were made specifically for studying rather than for looks. State is encoded in form as well as in value, so an HTTP 5xx, a 4xx and a 2xx are visually distinct at a glance, as are ERROR, WARN and INFO, and blocked against allowed traffic. Series colours are held at similar lightness and kept clear of the green interface accent, so no series shouts and no chart series can be mistaken for a piece of interface.

Charts are drawn as inline SVG with no charting library, since adding one would have meant a dependency the environment could not fetch.

Packaging is deliberately primitive. The build script is file concatenation:

```
.\build.ps1
-> dist\index.html      standalone, opens by double-click, works offline
-> dist\artifact.html   same content without the document wrapper
```

There is no bundler, no transpiler and no dependency tree, which is what allows the result to be a single file that will still open in ten years.

8. Deliberate limits

The following are out of scope. The application reports a clear message when one is reached rather than failing obscurely.

erex, which generates a regular expression from examples, is documented but not implemented.

The exam tests rex.

outputlookup is accepted but writes nothing, since there is no persistent store.

map is capped at ten input rows to prevent runaway subsearch execution.

Data models, pivot, tstats and accelerated searches are not modelled.

The hashing functions return deterministic stand-in values rather than real digests.

Full-text search matches on token boundaries rather than reproducing Splunk segmentation rules in full, so unusual punctuation may differ.

9. Summary

Measure	Value
Source files	5 JavaScript, 1 CSS, 1 build script
Shipped artefact	One HTML file, approximately 258 KB
Runtime dependencies	None
SPL commands implemented	About 60
Generated events	5,970 across 6 indexes and 6 sourcetypes
Lookup tables	5
Graded exercises	25, marked by result comparison
Assertions in the test suite	About 80, plus every documented example
Typical search time	Around 30 ms

The engine is the deliverable. The interface, the sample data and the exercises exist to give it something to be correct about, and the verification exists because an SPL interpreter that is wrong in silence would teach the wrong thing with complete confidence.